44

Chapitre 2

Introduction au Développement Android



Naoufel KHAYATI
naoufel.khayati@eniso.u-sousse.tn

AU. 2025 - 2026

44

GooglePlex

45

- Le siège social de Google se situe à Mountain View en Californie, au cœur de la "Silicon Valley", non loin de San Francisco, aux Etats-Unis.
  - L'OS Android est développé au bâtiment n°44.



- La mascotte Android
  - Bugdroid



Site : **http://developer.android.com**

45

PLAN

46

- Qu'est ce qu'Android ?
- Versions
- Outils nécessaires
- Null Safety du langage Kotlin
- Structure d'un projet
- Ressources Android

46

Qu'est ce qu'Android ?

47

- Un OS open source pour Smartphones, tablettes, TV et tout autre type de terminaux mobiles.
- Conçu par « Android », une startup rachetée par Google en 2005.
- Annoncé officiellement le 5 novembre 2007.
- Conçu pour intégrer au mieux les applications de Google.
- Il est fourni sous licence open source.

47

Versions

48

- Les différentes versions d'Android ont toutes des noms de desserts et suivent une logique alphabétique.

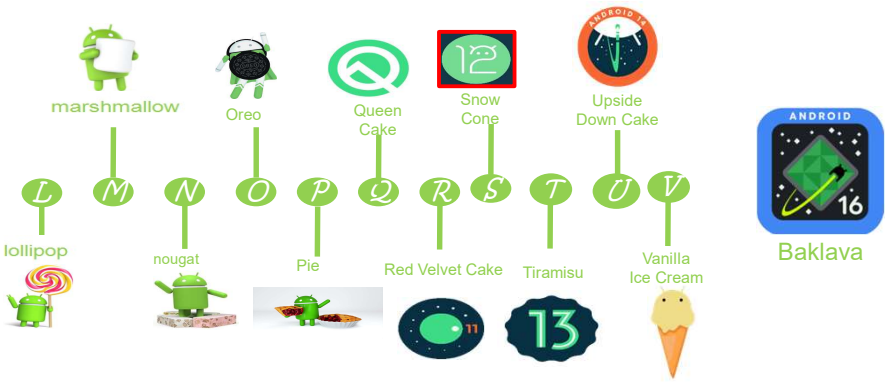


48

Versions

49

- Les différentes versions d'Android ont toutes des noms de desserts et suivent une logique alphabétique.



49

Versions (disparues)

50

- Fin 2007
  - 1.0 : *Apple Pie* (Tarte aux pommes), version connue uniquement ou presque des développeurs car c'est la version du SDK distribuée avant la sortie du premier téléphone Android;



- 2008
  - 1.1 : *Banana Bread* (Cake à la banane) ou Banana Split, version Béta incluse dans le téléphone HTC G1/Dream ;



- 2009
  - 1.5 : *Cupcake* (Petit Gâteau)
  - 1.6 : *Donut* (Beignet)



50

Versions (disparues)

51

- 2010
  - 2.1 (API 7) : *Éclair*

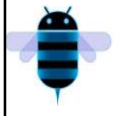




  - 2.2 (API 8) : *FroYo* (*Frozen Yogourt* : Yaourt glacé)
  - 2.3 (2.3.3) (API 9/10) : *Gingerbread* (Pain d'épice), dernière version dédiée uniquement aux Smartphones



- 2011



  - 3.0 (3.1/3.2) (API 11/12/13) : *Honeycomb* (Rayon de miel), version pour grandes tablettes et télévisions connectées
  - 4.0 (4.0.3) (API 14/15) : *Ice Cream Sandwich* (Sandwich à la crème glacée), version unifiée pour Smartphone, tablette et Google TV



51

Versions (disparues)

52

- 2012
  - 4.1 (4.1.2) (API 16) : *Jelly Bean (Dragée)*, version annoncée le 09 juillet 2012.
  - 4.2 (4.2.2) (API 17) : *Jelly Bean*, version annoncée le 13 novembre 2012.



- 2013
  - 4.3 (API 18) : Jelly Bean, version annoncée le 24 juillet 2013.
  - *4.4(.2) (API 19)*: KitKat, le 4 novembre 2013.



52

Versions

53

- 2014
  - 4.4.W (API 20) : Kitkat, versions sorties en juin 2014.
  - 5.0(.1) (API 21) : Lollipop, en novembre 2014.





- 2015
  - 5.1.1 (API 22) : Lollipop, en mars /avril 2015
  - 6.0 (API 23) : Marshmallow, en août 2015.



- 2016
  - 7.x (API 24/25) : Nougat, en août 2016



53

Versions

54

- 2017
  - 8.x (API 26/27) : Oreo, en août / décembre 2017.



- 2018
  - 9.x (API 28) : Pie, en mars 2018.



- 2019 : Android 10 – Q (API 29) : 3 septembre 2019.



- 2020
  - Android 11 (API 30) : 8 septembre 2020.



54

Versions

55

- 2021
  - Android 12 (API 31/32) : 4 octobre 2021.



- 2022
  - Android 13 (API 33) : 15 Août 2022 sur les appareils Google Pixel.



- 2023
  - Android 14 (API 34) : Octobre 2023.



- 2024
  - Android 15 (API 35) : Septembre 2024.



- 2025
  - Android 16 (API 36) : Juin 2025.



55

Versions

56

- Répartition des différentes versions en avril 2025 selon Google.

Android version	Percentage
15 / V	4.5%
14 / U	27.4%
13 / T	16.8%
12 / S	12.8%
11 / R	15.9%
10 / Q	10.2%
9 / Pie	5.8%
8.1 / Oreo	3.0%
8 / Oreo	1.0%
7.1 / Nougat	0.6%
7 / Nougat	0.6%
6 / Marshmallow	0.7%
5.1 / Lollipop	0.5%
5 / Lollipop	0.1%
4.4 / KitKat	0.1%

56

Outils nécessaires

57

- Java / Kotlin





- JRE
- Android Studio



  - IDE (IntelliJ IDEA)
  - Android SDK
    - *https://developer.android.com/studio*

57

Outils nécessaires – Langages

58

- **JAVA** : le premier langage adopté par Google pour le développement Android.
- **KOTLIN,** développé par l'entreprise JetBrains connue par son IDE « IntelliJ »
  - Langage **concis** et **100 % interopérable** avec Java et sa JVM.
  - Intègre de nouvelles fonctionnalités et améliorations en termes de sécurité et performance (par rapport à Java) telle que « **Null Safety** » (*Voir slides 60 - 63*).
  - En mai 2017, Google annonce Kotlin comme étant officiellement supporté sur la plateforme Android.

58

Modes de Développement

59

- **XML Views** : Développement d'applications à base d'interfaces XML.
  - Code métier dans des classes Kotlin ou Java.
  - Interface graphique dans des fichiers XML
    - Pour les XML Views, nous avons toujours le choix d'utiliser l'un ou l'autre. Mais, l'utilisation de Kotlin est recommandée par Google.
- **Jetpack Compose** : Développement entier dans les classes Kotlin.
  - Pour Jetpack Compose, seul Kotlin est utilisé.

❖ Dans ce cours, nous allons utiliser la première option avec le langage Kotlin.

59

Null Safety

60

- Sert à éviter ***NullPointerException.***
- Par défaut, les types exposés par les langages de programmation sont non nullables (leurs valeurs ne doivent pas être ***null***).
  - Erreur de compilation

```
var s1 = "Android"
s1 = null
```

Null can not be a value of a non-null type String

Add 'toString()' call Alt+Shift+Enter More actions... Alt+Enter

```
var s1 : String = "Android"
s1 = null
```

60

Null Safety

61

- Kotlin ajoute la notion des types nullables (String?, Int?, Float?, etc.).

```
var s2 : String? = "Android"
s2 = null
```

```
var l1 = s1.length

var l2 = s2.length
```

Only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver of type String?

Replace with safe (?.) call Alt+Shift+Enter More actions... Alt+Enter

```
var l2 = s2?.length
```

61

Null Safety

62

- Avec l'utilisation du « Safe Call Operator » (?), une seule ligne remplace toute une instruction conditionnelle.

```
var l2 = s2?.length
```

```
var l2:Int?
if (s2!=null)
    l2=s2.length
else
    l2=null
```

```
var l2 = if (s2 != null)
             s2.length
         else
             null
```

```
var l2 =s2?.length
    var l2: Int?
```

62

Null Safety

63

- L'utilisation de l'opérateur « !! » (Non-null Assertion Operator), indique au compilateur que le développeur affirme explicitement que la valeur de s2 est non nulle à cet endroit.
  - La longueur de s2 sera bonne si s2 est non nulle, sinon une exception sera levée.

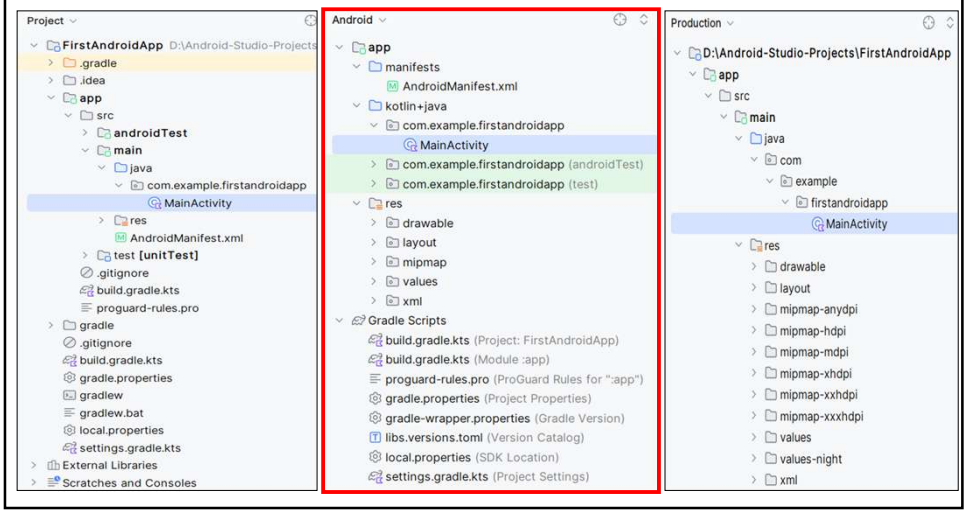
```
var l2 =s2!!.length
    var l2: Int
```

63

Structure d'un projet

64

- Différentes vues d'un projet (Project, Android, Production, Packages, etc.)



64

Structure d'un projet – Le dossier *java*

65

- **java** : contient le paquetage de l'application et ceux des tests.
  - Le paquetage de l'application contient les fichiers sources .kt des activités (ici, un seul fichier MainActivity.kt)

```kotlin
package com.example.firstandroidapp

import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
    }
}
```

65

Structure d'un projet – Le dossier *res*

66

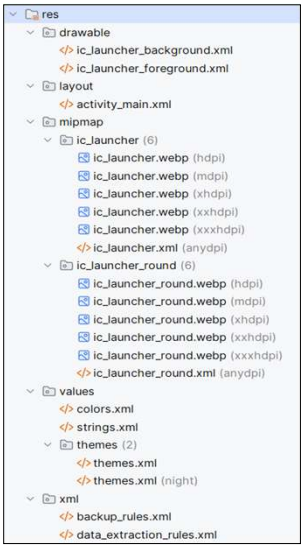
- **res** : contient les ressources de l'application, càd les éléments qui s'afficheront à l'écran ou qui influenceront ce qui s'affichera à l'écran.
- Les ressources sont des éléments capitaux dans une application Android.
  - On y trouve par exemple des chaînes de caractères, des couleurs, des tailles, des images, des fichiers XML de layout, des fichiers XML de menus, etc.

66

Structure d'un projet – Le dossier *res*

67

- Les ressources sont de plusieurs types.
- Pour permettre à Android de les retrouver facilement, chaque type de ressources est associé à un répertoire particulier.
  - Aucune ressource ne doit être placée directement dans res/
    - Erreur de compilation



67

Structure d'un projet – Le dossier *res*

68

- **res/drawable**: Dessins et images
  - On y trouve les images matricielles (PNG, JPEG, GIF).
  - On peut y trouver aussi des fichiers XML dont le contenu décrit des formes ou des dessins.
- **res/mipmap**: Images aussi, qui servent comme des icônes, par exemple.
  - Une image placée dedans, se verra avoir différentes densités (mdpi, hdpi, xhdpi, etc.) pour s'adapter à différentes résolutions d'écrans.

68

Structure d'un projet – Le dossier *res*

69

- **res/layout**: Mise en page d'écran ou interface graphique (gabarit)
  - Les fichiers XML qui représentent la disposition que doivent adopter les vues.
- **res/menu** (n'apparaît pas dans notre exemple): Pour les menus
  - Les fichiers XML décrivant des menus.
- **res/values**: Données
  - Les ressources qui appartiennent à cette catégorie sont nombreuses. On y trouve entre autres des chaines de caractères, des dimensions, des couleurs, etc.
- **res/xml**: Un emplacement polyvalent pour divers fichiers XML utilisés pour la configuration, les préférences, les animations et toutes autres ressources personnalisées.

69

Structure d'un projet – Le dossier *manifests*

70

- **manifests:** Contient le fichier **AndroidManifest.xml** qui est généré dès la création du projet.
  - Il ne doit être ni renommé ni déplacé.
- Il déclare et décrit l'ensemble des éléments de l'application:
  - Nom et package de l'application, son icône, thème, etc.
  - Activités, Services, Intents
    - le point d'entrée de l'application
    - quels composants constituent ce programme;
  - Permissions nécessaires à l'exécution du programme
    - Accès à Internet, accès à l'appareil photo, appels téléphoniques, etc.

70

Structure d'un projet – Le dossier *manifests*

71

- Le fichier **AndroidManifest.xml**

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="FirstAndroidApp"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.FirstAndroidApp"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

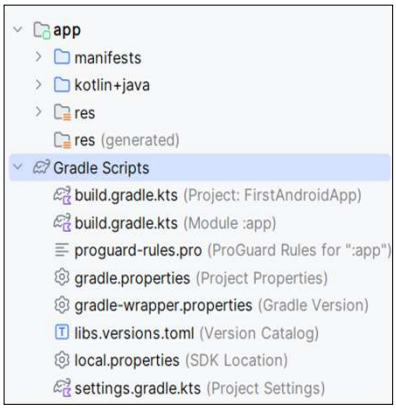
</manifest>
```

71

Structure d'un projet – Les scripts *Gradle*

72

- **Gradle Scripts**
  - Android Studio se base sur le système de build «GRADLE ».
    - Outil de génération de projets comme Make, Ant et Maven.
  - De même que Make se sert d'un fichier Makefile, Gradle se sert d'un fichier nommé build.gradle.



72

Structure d'un projet – Les scripts *Gradle*

73

- Sous Android Studio, il y a deux *build.gradle* : (1)
  - Le premier script dans le dossier racine du projet.
  - Il sert à indiquer la configuration commune à tous les modules du projet (dépendances générales, plugins généraux, etc.).

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
}
```

73

Structure d'un projet – Les scripts *Gradle*

74

- Sous Android Studio, il y a deux *build.gradle* : (2)
  - Le deuxième script dans le dossier 'app' pour la configuration du module (configuration de la compilation, dépendances et plugins spécifiques).

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
}

android {
    namespace = "com.example.firstandroidapp"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.firstandroidapp"
        minSdk = 24
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile( name: "proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_1_8
        targetCompatibility = JavaVersion.VERSION_1_8
    }
```

74

Structure d'un projet – Les scripts *Gradle*

75

- Généralement, ces deux fichiers *build.gradle* peuvent être mis à jour dans les cas suivants :
  - Changer la version de l'API Android de développement
  - Changer la version de l'outil de compilation
  - Changer la version minimale de l'API supportée par l'application
  - Appeler des bibliothèques externes
  - Etc.

75

Ressources – La classe R

76

- Comme Android est destiné à être utilisé sur une grande variété de supports, il fallait trouver une solution pour :
  - Permettre à une application de s'afficher de la même manière sur différentes tailles et résolutions d'écrans.
  - Faire en sorte que les textes s'adaptent à la langue de l'utilisateur.
- Pour cela, Android organise les différentes ressources d'une application d'une manière très précise, de façon qu'il sache quels éléments utiliser pour quels types de terminaux.
- Ces ressources sont utilisées dans le code à travers la classe R.
  - Android régénère automatiquement cette classe à chaque changement dans le projet.
  - Toutes les ressources sont accessibles à travers R, dès qu'elles sont déclarées dans le fichier XML ou que le fichier associé est déposé dans le dossier adéquat.

76

Ressources – La classe R

77

- Pour les versions Android Studio 3.6 et plus, AGP (Android Gradle Plugin) génère désormais directement le bytecode de la classe R, au lieu du fichier R.java.
  - On ne trouvera plus le fichier R.java dans l'arborescence du projet.
  - ***NB : les quelques slides suivants vous donneront une idée sur cette classe ainsi que sur ses classes internes afin de comprendre l'utilisation des ressources qui y sont déclarées.***

77

Ressources – La classe R

78

- La classe R contient des classes en membres statiques pour chaque type de ressources défini dans le projet.
- Chaque sous-classe expose au moins une variable dont le nom correspond à l'identifiant de la ressource.
- Pour utiliser une ressource, il faut connaître son type et son identifiant.

R.java

Files under the "build" folder are generated and should not be edited.

```
1     /* AUTO-GENERATED FILE. DO NOT MODIFY.
2      *
3      * This class was automatically generated by the
4      * aapt tool from the resource data it found. It
5      * should not be modified by hand.
6      */
7
8     package com.example.chapitre3_2019;
9
10    public final class R {
11      public static final class anim {...}
25      public static final class attr {...}
3088    public static final class bool {...}
3093    public static final class color {...}
3182    public static final class dimen {...}
3298    public static final class drawable {...}
3400    public static final class id {...}
3550    public static final class integer {...}
3557    public static final class layout {...}
3598    public static final class mipmap {...}
3602    public static final class string {...}
3646    public static final class style {...}
3955    public static final class styleable {...}
13819 }
```

78

Ressources – La classe R

79

La classe « layout »

- Une classe **interne**, pour y accéder, il faut faire référence à la classe qui la contient (R).

```
public static final class layout {
public static final int activity_main=0x7f09001c;
}
```

- Le mot clé static, dans le cas d'une classe interne, signifie que la classe n'est pas liée à une instanciation de la classe qui l'encapsule.
  - Pour accéder à **layout**, on ne doit pas nécessairement créer un objet de type **R**.
  - On peut y accéder par **R.layout**
- Le mot clé final signifie que l'on ne peut pas créer de classe dérivée de layout.

79

Ressources – La classe R

80

- Cette classe contient des entiers « public », « static » et « final».
  - Dans notre exemple, on ne montre qu'un seul entier.
  - Il s'agit d'une constante, à laquelle n'importe quelle autre classe peut accéder sans avoir à créer d'objet de type layout ni de type R.
  - Cet entier est sous la forme d'un hexadécimal sur 32 bits.
  - Cet entier a le même nom qu'un fichier de ressources, tout simplement parce qu'il représente ce fichier XML.
    - Cet entier est un identifiant unique pour le fichier de mise en page qui s'appelle *activity_main.xml*.
  - L'identifiant d'un layout est accessible comme suit:

Java / Kotlin

```
R.layout.activity_main
```

XML

```
"@layout/activity_main"
```

80

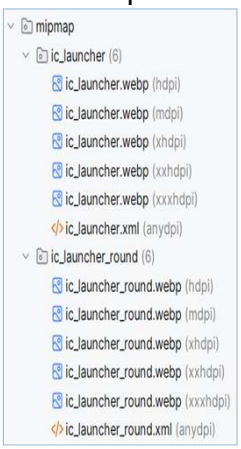
Ressources – La classe R

81

La classe mipmap

- Contrairement au cas précédent, on a un seul entier pour plusieurs fichiers qui ont le même nom.

```
public static final class mipmap {
    public static final int ic_launcher=0x7f0a0000;
    public static final int ic_launcher_round=0x7f0a0001;
}
```



- Quand on fera appel à l'identificateur, Android saura le fichier qu'il lui faut (par exp. *ic_launcher*) et déterminera automatiquement quel est le répertoire le plus adapté au matériel selon la résolution de l'écran.
- De même, l'identifiant d'un mipmap est accessible comme suit:

Java / Kotlin

```
R.mipmap.ic_launcher
```

XML

```
"@mipmap/ic_launcher"
```

81

Ressources – La classe R

82

La classe String

- Cette fois, pour toute chaîne de caractères dans le fichier *res/values/strings.xml* correspond un entier dans la classe string.

```
public static final class string {
  public static final int app_name=0x7f0b0027;
  public static final int course=0x7f0b0028;
  public static final int hello_world=0x7f0b0029;
}
```

- L'identifiant d'une chaine de caractères est accessible comme suit:

Java / Kotlin

```
R.string.hello_world
```

XML

```
"@string/hello_world"
```

82

Ressources – La classe R

83

Autres valeurs simples

- Plusieurs fichiers XML peuvent être placés dans **res/values.**
- Cela permet de définir des couleurs, des dimensions, des tableaux (voir exemple + répercussion sur R.java), etc.



83

Ressources – Organisation

84

- Les ressources peuvent être gérées en fonction du matériel.
- Les emplacements indiqués précédemment sont ceux par défaut, mais il est possible de définir d'autres emplacements pour préciser des détails afin de se conformer au matériel de l'utilisateur / à la langue de l'OS.
  - Et si Android ne trouve pas d'emplacement dont le nom correspond exactement aux spécifications techniques du terminal ???
    - il cherchera parmi les autres répertoires qui existent, la configuration la plus proche.

84

Ressources – Organisation

85

- En partant du nom du répertoire par défaut, on peut créer d'autres dossiers qui permettent de préciser à quels types de matériels les ressources de ce répertoire sont destinées.
  - Il y a une syntaxe très précise à respecter pour qu'Android sache dans quel répertoire il doit fouiller.
  - Les restrictions sont représentées par des qualificateurs, au nombre de 20, qui permettront de préciser le matériel de destination.

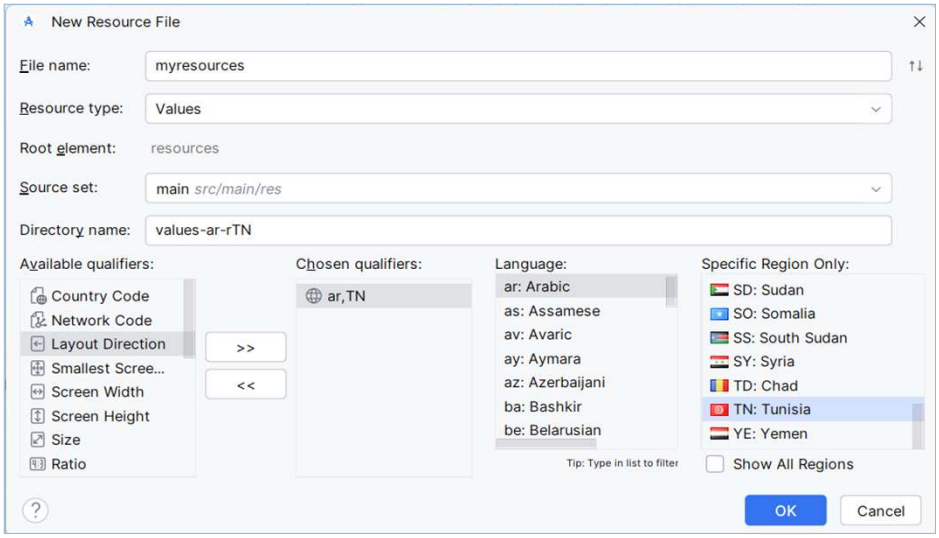
res/<type_ressource>[<-qualificateur1><-qualificateur 2>…<-qualificateur N>]

  - Ces qualificateurs doivent être ordonnés selon leurs priorités.

85

Ressources – Organisation

86



86

Ressources – Organisation

Quelques qualificateurs

87

- **Langue et région (Locale : Priorité 3)**
  - La langue préférée de l'utilisateur.
  - On indique une langue puis, éventuellement, on peut préciser une région avec « -r ».
  - Exemples :
    - « en » pour l'anglais ;
    - « fr » pour le français ;
    - « fr-rFR » pour le français mais uniquement celui utilisé en France ;
    - « fr-rCA » pour le français mais uniquement celui utilisé au Québec ;
    - etc.
- **Taille de l'écran (Size : Priorité 8)**
  - Il s'agit de la longueur de la diagonale de l'écran (small, normal, large, xlarge)

87

Ressources – Organisation

Quelques qualificateurs

88

- **Orientation de l'écran (Orientation : Priorité 10)**
  - Portrait / Landscape (Paysage).
- **Résolution de l'écran (Density : Priorité 13)**
  - Low density : environ 120 dpi
  - Medium density : environ 160 dpi
  - High density : environ 240 dpi
  - X-High density : environ 320 dpi
  - XX-High density : environ 480 dpi
  - No density : pour ne pas redimensionner les images matricielles.
- **Version d'Android (Version : Priorité 20)**
  - Il s'agit du niveau de l'API (v3, v4, v7, v19, v20, etc.).

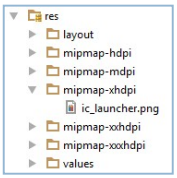
88

Ressources – Organisation

Quelques qualificateurs

89

- Exemples
  - **res/mipmap-xhdpi** pour avoir des images spécifiquement pour les écrans à très haute densité.



  - **res/layout-fr-rFR** pour avoir une mise en page spécifique destinée à ceux qui ont choisi la langue « Française de France ».
  - **res/values-fr-rCA-port** pour des données qui s'afficheront uniquement à ceux qui ont choisi la langue « Française du Québec » et dont le téléphone se trouve en orientation portrait.

Une mise en page destinée à la version 25, quand l'utilisateur penche son téléphone en mode portrait alors qu'il utilise le Français des Belges et que son terminal a une résolution moyenne.

res/layout-fr-rBE-port-mdpi-v25

89

Pour finir… Exemple d'application multilingue

90

- **Exemple Utile** : traduire les chaines d'une application sans la recompiler et sans la développer 2 fois ou plus.
  - Dans ***res/values/strings.xml*** (dossier par défaut), on définit les chaînes de l'application (ici en Anglais), au lieu de les mettre comme des constantes dans le code source.

```
<resources>
    <string name="app_name">FirstApp</string>
    <string name="hi">Hello</string>
    <string name="main_menu">Main Menu</string>
    <string name="config">Settings</string>
</resources>
```

90

Pour finir… Exemple d'application multilingue

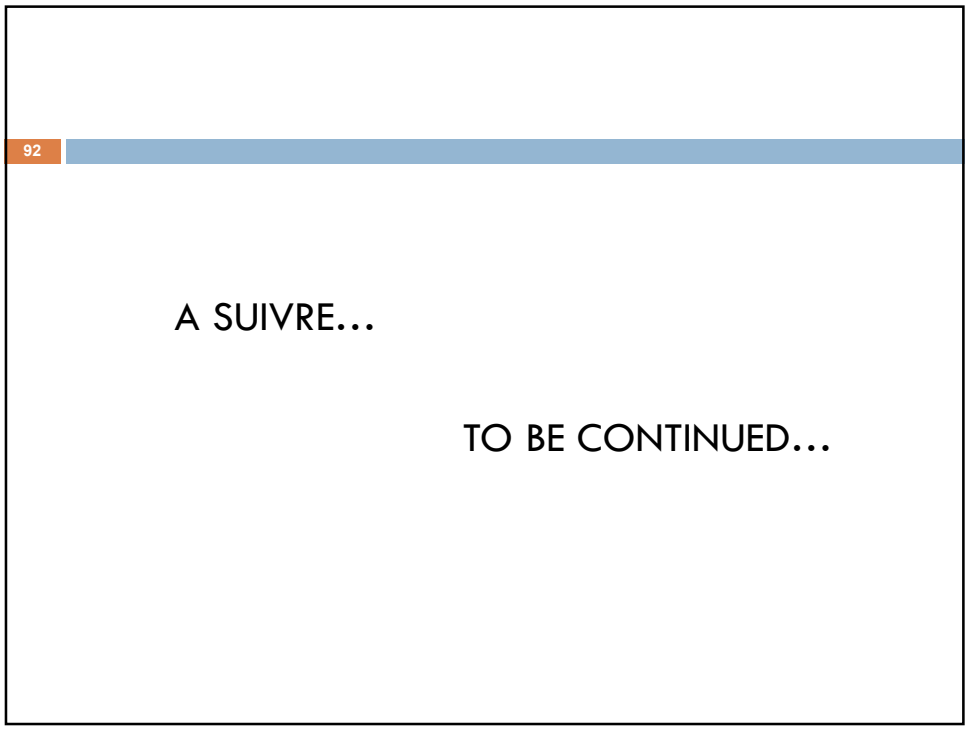
91

- Puis, il suffit de faire des copies du contenu de ce fichier dans les dossiers ***values-fr***, ***values-it***, ***values-de***, etc. et de traduire les textes tout en gardant les attributs '**name**'.
- Voici par exemple ***res/values-fr/strings.xml*** :

```
<resources>
    <string name="app_name">FirstApp</string>
    <string name="hi">Salut</string>
    <string name="main_menu">Menu Principal</string>
    <string name="config">Configuration</string>
</resources>
```

- Le système Android ira chercher automatiquement le bon texte en fonction des paramètres linguistiques configurés par l'utilisateur.

91



92

Naoufel KHAYATI